



POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

Best Bike Paths: Design Document (DD)

Software Engineering II

Advisor: Prof. Rossi Matteo Giovanni

Students:

- Lorusso Luca (10831296)
- Martorano Gabriele (10822443)

Version: 1.0

Date: 07-01-2026

Content

Content.....	2
List of Figures.....	4
List of Tables.....	5
1. Introduction.....	1
1.1. Purpose.....	1
1.2. Scope.....	1
1.3. Definitions, Acronyms and Abbreviations.....	2
1.4. Revision History.....	2
1.5. Reference Document.....	2
1.6. Document Structure.....	3
2. Architectural Design.....	4
2.1. Overview.....	4
2.2. Component View.....	5
2.3. Deployment View.....	10
2.4. Runtime View.....	12
2.5. Components Interfaces.....	16
2.6. Data Logic Model.....	18
2.7. Selected Architectural Style and Patterns.....	19
2.7.1. 3-Tier Architecture.....	19
2.7.2. Model-View-Controller (MVC) pattern.....	19
2.7.3. Service-Oriented Decomposition within the Application Tier.....	20
3. User Interface Design.....	21
3.1. Public Pages.....	22
3.1.1. Public Pages Navigation Map.....	22
3.1.2. Login Pages Mock-up.....	22
3.1.3. Registration Page Mock-up.....	23
3.1.4. Search Path Pages Mock-up.....	24
3.2. Private Pages.....	25
3.2.1. Private Pages Navigation Map.....	25
3.2.1. Home Page Mock-up.....	26
3.2.2. Trip Recording Pages Mock-up.....	26
3.2.3. Manual Report Pages Mock-up.....	28
3.2.4. Review Auto-Detected Issues Pages Mock-up.....	29
3.2.5. Unpublished Reports Page Mock-up.....	31
3.2.6. Profile Settings Page Mock-up.....	32
3.2.7. Utility Pages Mock-up.....	32
3.3. Requirements UI Sections Mapping.....	33
4. Requirements Traceability.....	35
4.1. Registered User Goals.....	35
4.2. Guest User Goals.....	36
5. Implementation, Integration and Test Plan.....	37

5.1. Overview.....	37
5.2. Implementation Plan.....	37
5.2.1. Feature Identification.....	37
5.3. Component Integration and Testing.....	38
5.4. System Testing.....	39
6. Effort spent.....	40
7. References.....	41
7.1. References.....	41
7.2. Used Tools.....	41

List of Figures

- 2.1. High-level Architecture
- 2.2. System UML Component Diagram
- 2.3. Auto Detection System UML Component Diagram
- 2.4. Report and Validation System UML Component Diagram
- 2.5. Routing & Map Integration System UML Component Diagram
- 2.6. Recommendation & PathScore Engine UML Component Diagram
- 2.7. UML Deployment Diagram
- 2.8. Secure Login - Sequence Diagram
- 2.9. Start/Stop Trip Recording - Sequence Diagram
- 2.10. Review Auto-Detected Issues - Sequence Diagram
- 2.11. Search and View Paths - Sequence Diagram
- 2.12. Manual Data Contribution - Sequence Diagram
- 2.13. View Trip Details - Sequence Diagram
- 2.14. Publish Validated Information - Sequence Diagram
- 2.15. Data Logic Model Scheme
 - 3.1. Public Pages Navigation Map
 - 3.2. WelcomePage Mock-up
 - 3.3. LoginPage Mock-up
 - 3.4. RegistrationPage Mock-up
 - 3.5. SearchPathPage Mock-up
 - 3.6. PathResultPage Mock-up
 - 3.7. PathDetailsPage Mock-up
 - 3.8. Private Pages Navigation Map
 - 3.9. HomePage Mock-up
 - 3.10. TripRecordingPage Mock-up
 - 3.11. TripSummaryPage Mock-up
 - 3.12. TripHistoryPage Mock-up
 - 3.13. ManualReportFormPage Mock-up
 - 3.14. InvalidReportMessagePage Mock-up
 - 3.15. DetectedReportsListPage Mock-up
 - 3.16. EditReportPage Mock-up
 - 3.17. UnpublishedReportsListPage Mock-up
 - 3.18. ProfileSettingsPage Mock-up
 - 3.19. LoadingPage Mock-up
 - 3.20. ErrorMessagePage Mock-up

List of Tables

- 1.1. Definitions, Acronyms and Abbreviations table
- 3.1. Requirements UI Sections Mapping
- 6.1. Member Effort Spent table

1. Introduction

1.1. Purpose

In modern cities, due to increasing traffic congestion and growing attention to sustainable mobility, many people choose bicycles for commuting and leisure activities. However, cyclists often face difficulties in identifying safe, efficient, and well-maintained routes, since common navigation applications usually lack updated and detailed information about the actual conditions of bike paths.

To support cyclists in this context, a new platform called Best Bike Paths (BBP) is to be developed. BBP allows users to search for and visualize bike paths between a chosen origin and destination, providing information about path conditions, obstacles, and overall route quality. The system relies on both user-generated data and automatically collected information to build a shared and reliable representation of the bike path network.

1.2. Scope

As described in the RASD, there are two main categories of users that interact with the BBP system: Registered Users and Guest Users.

Registered Users actively contribute to the platform by recording bike trips and providing information about bike paths. They can manually insert data regarding the condition of specific path segments or report obstacles, specifying their position and characteristics. In addition, Registered Users may activate an automated mode that allows the system to collect data from the device's GPS, accelerometer, and gyroscope in order to detect potential anomalies along the path. Automatically detected information must be reviewed and confirmed or corrected by the user before it can be published and made available to the community.

Guest Users, on the other hand, interact with the system only to consult publicly available information. They can search for bike paths between an origin and a destination and visualize alternative routes on a map, comparing them based on a computed Path Score that reflects the effectiveness and condition of the paths. Guest Users do not provide data but benefit from the information shared by the community.

Once path information is validated and published, it becomes part of the shared bike path inventory used by the system to rank and recommend routes. The BBP system integrates external services to support its functionalities, such as an External Weather Service to enrich trip data with meteorological information and an External Map Service to display routes and reports on a map.

1.3. Definitions, Acronyms and Abbreviations

Acronym	Definition
ADS	Auto Detection System
API GW	Application Programming Interface Gateway
BBP	Best Bike Path
DBMS	Database Management System
DD	Design Document
EMS	External Map Service
EWS	External Weather Service
GPS	Global Positioning System
GU	Guest User
MC	Mobile Client
PSE	Recommendation and PathScore Engine
R&V	Report & Validation System
RASD	Requirements Analysis and Definition Specification
RMI	Route Management and Map Integration System
RU	Registered User
U&T	User & Trip Management System
UCx	Use Case number x
II&T	Implementation, Integration and Test Plan

Table 1.1: Definitions, Acronyms and Abbreviations table

1.4. Revision History

Version 1.0 (07-01-2026): Initial release.

1.5. Reference Document

- Specification document: Assignment RDD AY 2025-2026
- Requirements Analysis and Specification Document (RASD)
- Provided course materials and notes
- Design Document Reference Model

1.6. Document Structure

1. Introduction:

This section presents the overall purpose and objectives of the Design Document (DD), outlining its motivation and the scope of the architectural work. It formally defines the key terms, acronyms, and abbreviations used throughout the document, establishing a clear reference base.

2. Architectural Design:

This is the core of the document, providing a comprehensive design of the BBP system from an architectural perspective. It details the logical 3-Tier structure and presents various architectural views, including the Component View (detailing internal modules like the Auto Detection System and Recommendation & PathScore Engine), the Deployment View (mapping components to physical infrastructure), and the Runtime View (illustrating dynamic interactions). It also justifies the selection of the primary architectural style and relevant design patterns.

3. User Interface Design:

This section focuses on the non-functional attributes related to usability, detailing the design choices for the BBP application's front-end. It provides an overview of the key screens (e.g., Trip Recording, Path Search, and Data Review/Validation screens) and defines the user interface (UI) principles, ensuring that the critical data validation and map visualization requirements are met efficiently.

4. Requirements Traceability:

This section provides the essential link between the initial requirements and the developed design. It features the Traceability Matrix, which explicitly maps every functional requirement (R01-R21) and critical non-functional requirement (e.g., Security, Reliability) from the RASD to the specific architectural components and design choices that fulfill them.

5. Implementation, Integration, and Test Plan:

This chapter outlines the practical strategy for developing the BBP system. It defines the implementation technologies (e.g., programming language, frameworks), details the integration steps between the core modules and external services (EWS, EMS), and presents a high-level test plan, focusing specifically on how the complex validation logic and PathScore calculation will be verified.

6. Effort Spent:

This section summarizes the amount of work done by each team member, specifying their individual contribution to the preparation of this DD.

7. References:

This section lists all bibliographic references, documents, and materials cited or used in the development of the present specification.

2. Architectural Design

2.1. Overview

From an implementation perspective, it was decided to develop the Best Bike Paths (BBP) system as a mobile-oriented application, designed to run on smartphones and support continuous data acquisition during bike trips. For this reason, the architecture chosen for its development is a 3-tier architecture. This is a software design pattern that separates the application into three distinct layers:

- 1. Presentation Tier:** the user interface where users interact with the system, running on a mobile device and responsible for data visualization, user input and sensor data acquisition;
- 2. Application Tier:** the tier in which the core business logic is implemented, including path search, path scoring, validation of reported data and interaction with external services;
- 3. Data Tier:** the layer responsible for managing the storage, retrieval and persistence of all system data.

This architectural choice offers significant advantages in terms of scalability, maintainability, and modularity, allowing each layer to evolve independently while preserving a clear separation of concerns.

Going into more detail, the Application Tier represents the core of the BBP system. It exposes a set of APIs that handle user authentication, trip recording, data validation and publication, path search, and Path Score computation. This layer also manages the integration with external services, namely the External Weather Service (EWS) used to enrich trip data with meteorological information, and the External Map Service (EMS) used to retrieve the base map for visualization purposes.

Moreover, the data layer relies on a relational database to manage both the structured and semi-structured data of the system, which constitute the majority of the data utilized by the BBP platform. Structured data include user profiles, bike paths, path segments, recorded trips, and reported conditions or obstacles, together with their validation and publication status. The relational data model allows the system to explicitly represent the relationships among users, trips, path segments, and reports, ensuring consistency and integrity of the stored information.

Data acquired during trip recording, such as GPS traces and sensor readings, are strictly associated with individual trips and are accessed in a controlled manner. These data are persisted after trip completion and stored using relational structures that support flexible attributes, avoiding the need for additional database technologies. This design choice simplifies the overall architecture while guaranteeing reliable storage, access control, and efficient management of both private and shared information.

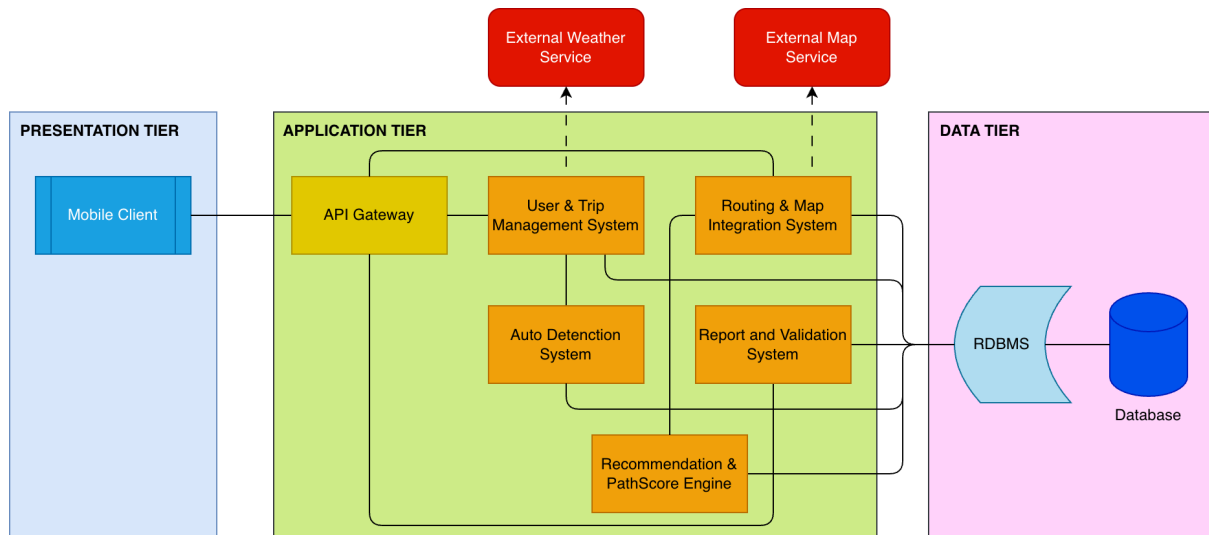


Figure 2.1: High-level Architecture

2.2. Component View

As mentioned, the BBP platform is developed using a 3-tier architecture approach. The Data Tier is implemented through a relational DBMS, which stores both structured entities (users, paths, reports) and trip-related data (trip metadata and recorded traces) through relational structures and/or flexible attributes. As for the back-end components, the following main macro-components can be identified:

- **User & Trip Management System**

It provides functionalities related to user access and the management of private trips, including trip creation, storage, visualization, and deletion. It is divided into several sub-components:

- **Authentication Manager:** It handles user authentication functionalities (registration, login, token/session management) and enforces role-based access control between Registered Users and Guest Users.
- **User Profile Manager:** It manages user profile data and preferences, including privacy-related settings that influence which trip elements may be included in published information.
- **Trip Lifecycle Manager:** It manages the lifecycle of a trip (start, pause, stop, completion), ensuring that trip data are correctly associated with the corresponding user and persisted at trip completion.
- **Trip Statistics Manager:** It manages the computation and storage of trip statistics (e.g., duration, total distance, average speed) and supports the retrieval of trip details in the user's private history.
- **Weather Adapter:** It is the component responsible for interfacing with the External Weather Service in order to enrich trips with meteorological information, handling failures and fallbacks as defined by the system requirements.

- **Auto Detection System**

It is the component responsible for processing sensor data acquired during trips in order to detect potential anomalies and propose candidate reports that must be

reviewed by the user before publication. It is divided into the following sub-components:

- **Sensor Data Preprocessor:** It receives trip traces and sensor readings, performs filtering/normalization steps, and prepares data for anomaly detection.
- **Anomaly Detection Engine:** It analyzes processed sensor data to detect potential obstacles or abnormal path conditions and generates candidate anomalies associated with specific path segments and positions.
- **Candidate Report Generator:** It creates draft/unconfirmed reports from detected anomalies, ensuring they are persisted with the appropriate state so that they can be reviewed by the user (mandatory validation step).

- **Report and Validation System**

It provides functionalities for managing all reports related to path conditions and obstacles, including manual insertion, user validation of automatically detected reports, and controlled publication. It is divided into the following sub-components:

- **Manual Report Manager:** It supports the creation of manually inserted reports (path conditions and obstacles), validating input consistency and associating reports to the correct path segments.
- **Review & Validation Manager:** It provides functionalities for reviewing, confirming, correcting, or rejecting automatically detected reports, enforcing the mandatory validation rule for automated data.
- **Publishing Manager:** It manages the publication workflow, ensuring that information becomes publicly accessible only upon explicit user command and only if the validation conditions are satisfied.
- **Report Editing Manager:** It supports post-publication editing or withdrawal of reports by their authors, enforcing ownership checks and maintaining consistency of public data.

- **Recommendation & PathScore Engine**

It is the component responsible for computing the Path Score and ranking candidate routes, combining path effectiveness with the impact of published reports. It is divided into the following sub-components:

- **Scoring Interface:** It retrieves from the database the data required for scoring (path segment metrics, published conditions, published obstacles) and prepares them for the scoring process.
- **PathScore Calculator:** It performs the Path Score computation for segments and routes according to the system's scoring model, applying penalties or adjustments based on the status and severity of published reports.
- **Ranking Manager:** It ranks candidate routes based on their computed scores and produces the ordered result set returned to the routing component.

- **Routing & Map Integration System**

It provides functionalities related to path search, route generation, and map visualization. It is divided into the following sub-components:

- **Routing Manager:** performs route search operations by interacting with the External Map Service to retrieve candidate routes, optionally applying user-provided filters.

- **Map Adapter:** It is the component responsible for interfacing with the External Map Service in order to retrieve the base map layer needed for visualization.
- **Overlay Generator:** It combines routing results with the PathScore and published reports, producing visualization-ready overlays (routes, markers, condition summaries) for the client.

- **API Gateway**

It is the component responsible for exposing the system functionalities to the mobile client and routing incoming requests to the appropriate internal macro-component. It is divided into the following sub-components:

- **Request Router:** It routes incoming REST requests to the corresponding internal services, enforcing the correct API structure.
- **Security Filter:** It validates authentication tokens, applies role-based access control, and performs basic request validation.
- **Response Assembler:** It aggregates and formats responses returned by internal services into client-ready payloads.

- **Other important components of the system**

In addition to the macro-components described above, the following components are part of the overall architecture:

- **Relational DBMS:** it stores both private and public data of the platform, including users, trips, paths, path segments, and reports with their states and publication flags.
- **External Weather Service (EWS):** a third-party service accessed through HTTPS to retrieve meteorological information used for trip enrichment.
- **External Map Service (EMS):** a third-party service accessed through HTTPS to retrieve the paths and base map layer for visualization.

- **Front-end components**

For the front-end, a single main client interacts with the back-end:

- **Mobile Client:** it represents the UI of the application running on the user's device. It interacts with the back-end through REST APIs exposed by the API Gateway. The Mobile Client also acquires sensor data (GPS, accelerometer, gyroscope) and computes real-time trip statistics locally, while persisting recorded data through the backend at trip completion.

The overall component diagram is shown in the following figure:

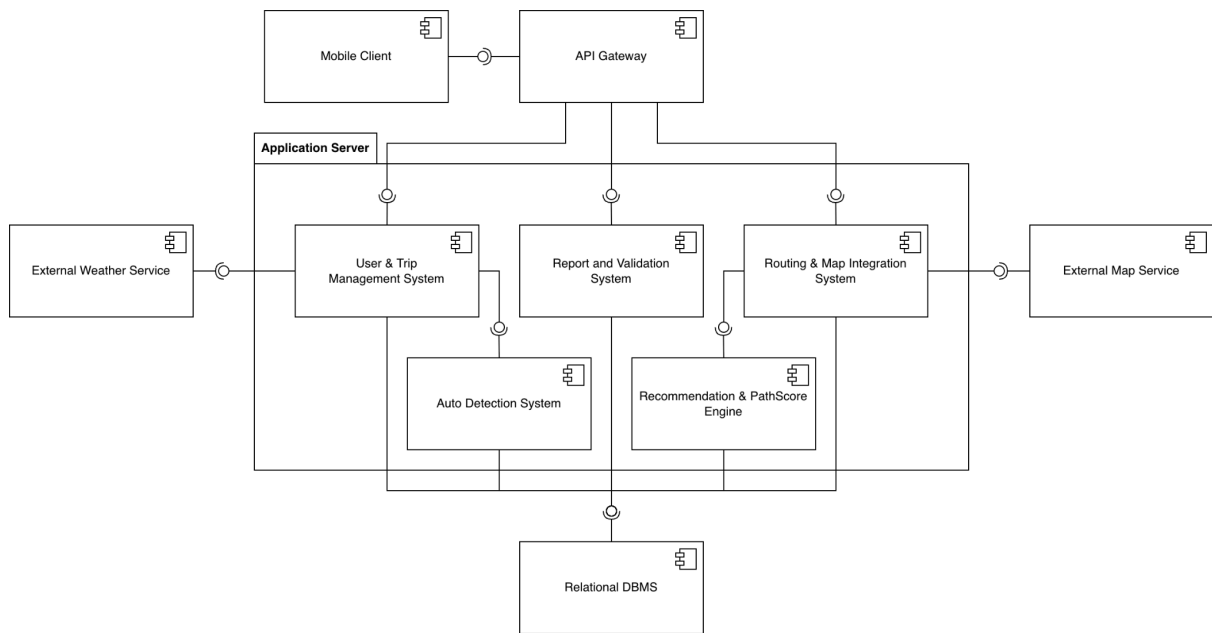


Figure 2.2: System UML Component Diagram

And here the detailed structure of each macro-component of the Application server:

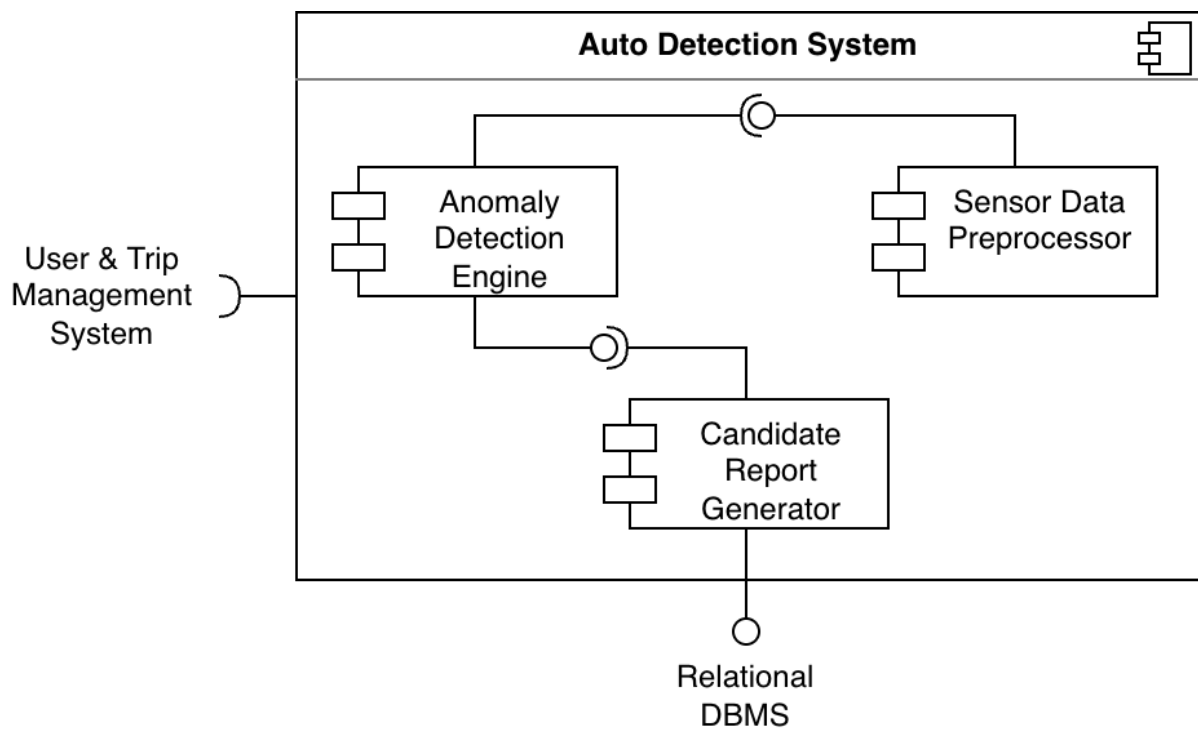


Figure 2.3: Auto Detection System UML Component Diagram

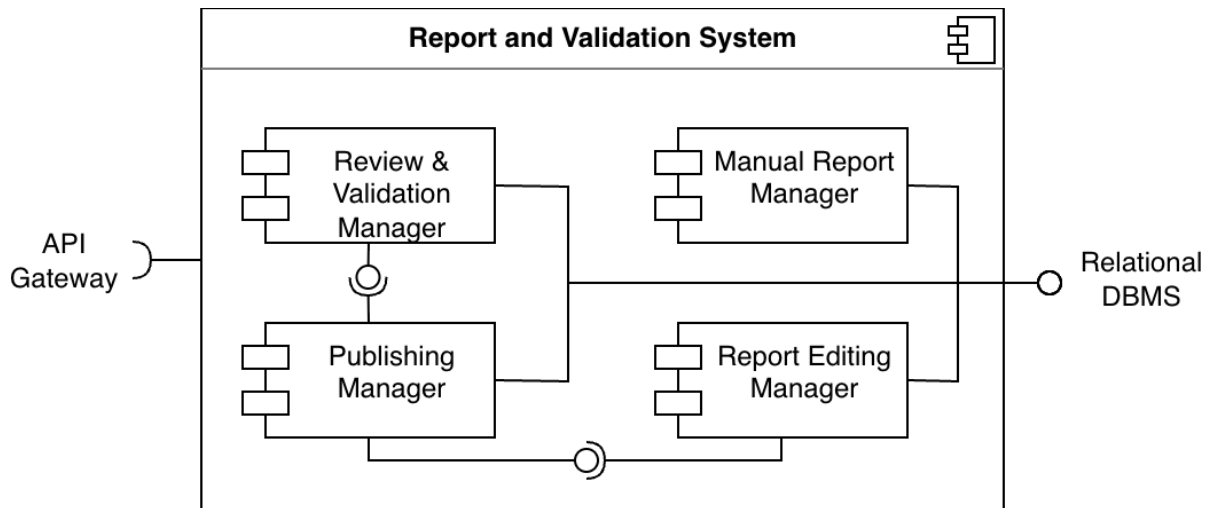


Figure 2.4: Report and Validation System UML Component Diagram

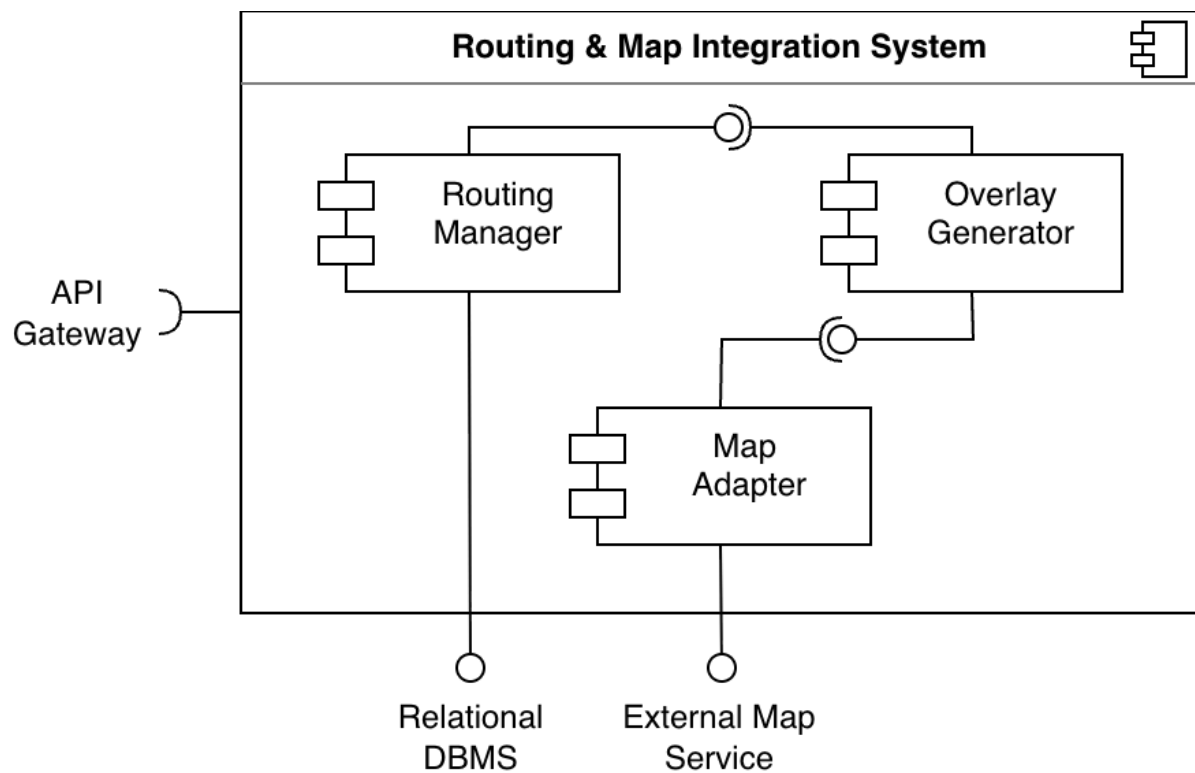


Figure 2.5: Routing & Map Integration System UML Component Diagram

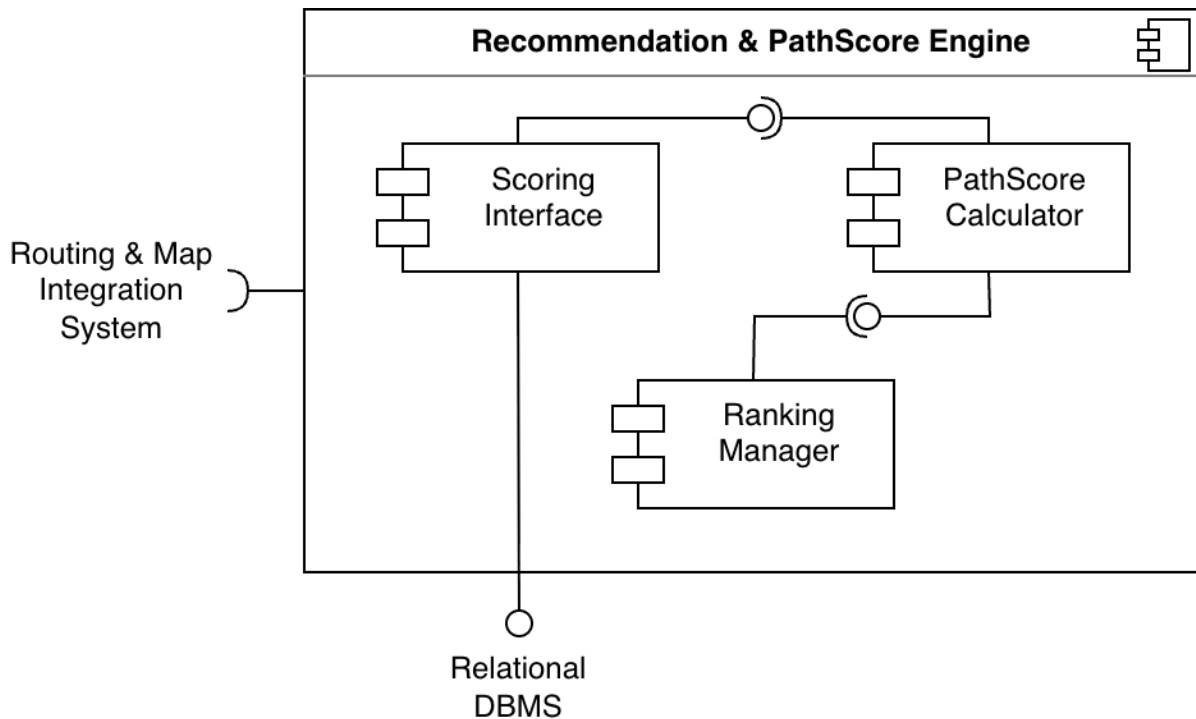


Figure 2.6: Recommendation & PathScore Engine UML Component Diagram

The Component View describes the internal structure of the Application Tier and the interactions among its main macro-components. Each component encapsulates a specific responsibility, ensuring separation of concerns and scalability. The API Gateway acts as the single entry point for client requests, while the remaining components handle user and trip management, anomaly detection and validation, path scoring, and routing and map integration.

2.3. Deployment View

With respect to system deployment, the proposed solution has been designed to satisfy the non-functional requirements stated in the RASD, particularly those related to availability, scalability, performance, security, and portability.

The BBP system adopts a three-tier architecture where the Presentation Tier is represented by the Mobile Client running on the user's mobile device, while the Application Tier and Data Tier are deployed on a cloud-based infrastructure.

The backend is composed of containerized services running on cloud-based virtual machines (e.g., an AWS EC2 environment), while persistent data are managed through a managed database service (e.g., a relational DBMS hosted on cloud storage infrastructure).

The resulting deployment architecture is illustrated in the following Deployment Diagram, which shows how the Presentation Tier, Application Tier, and Data Tier are distributed across cloud-based infrastructure and user devices.

- **Availability and scalability concerns**

- **Request distribution and fault tolerance:** Incoming requests from the Mobile Client are routed through an API Gateway, which represents the single entry point to the backend services. The API Gateway is deployed behind standard cloud security and networking services (e.g., firewalling, traffic filtering, and DDoS protection mechanisms such as AWS Shield).

A Load Balancer is placed in front of the Application Tier to distribute client requests across multiple backend instances. This design ensures continuous service availability and prevents single points of failure.

- **Horizontal scalability:** The server-side components and the database are deployed on a cloud infrastructure to ensure scalability, high availability, and reliable management of persistent data. With this choice the Application Tier can be replicated across multiple cloud-based virtual machines, allowing the system to scale horizontally according to workload peaks and the number of concurrent users.
- **Independent component scaling:** Backend components are deployed as independent containerized services, enabling selective replication and scaling based on specific workload characteristics.

- **Performance concerns**

During the design phase, particular attention was given to the deployment of computationally intensive components.

In particular, the PathScore Engine and the Auto Detection System are deployed within the AWS EC2-based application tier, but can be scaled independently on dedicated instances to isolate heavy computation from latency-sensitive requests. This architectural choice ensures that heavy computations—such as route scoring, ranking, and post-trip sensor data analysis—do not negatively impact latency-sensitive operations (e.g., authentication, path visualization, and report consultation).

It is important to note that real-time sensor acquisition and live trip statistics computation are handled locally by the Mobile Client on the user's device, while backend components operate exclusively on persisted trip data.

- **Security concerns**

- **Communication security:** All communications between the Mobile Client, the Application Tier, the Data Tier, and external services (e.g., External Weather Service and External Map Service) are enforced through HTTPS/TLS-secured channels.
- **Infrastructure protection:** The cloud deployment environment includes standard security mechanisms such as firewalling, network isolation, and traffic filtering, protecting backend services and sensitive user data.
- **Access control:** Authentication and authorization are enforced at the API Gateway and Application Server level, ensuring that only authorized requests can access protected functionalities reserved for Registered Users.

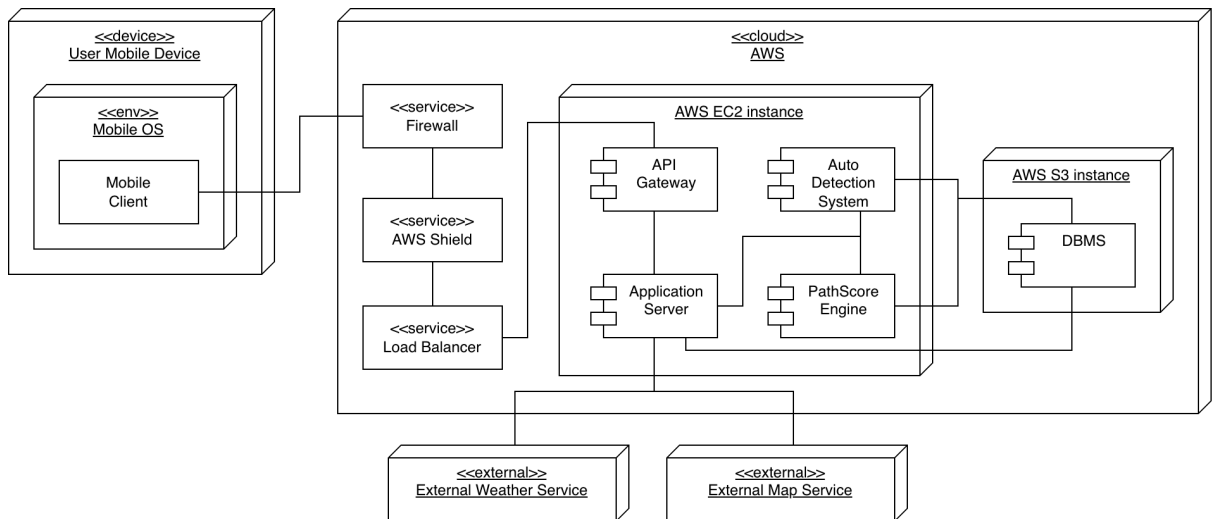


Figure 2.7: UML Deployment Diagram

2.4. Runtime View

The Runtime View describes the dynamic behavior of the BBP system by illustrating the interactions among the main components during the execution of the most significant use cases defined in the RASD.

The following sequence diagrams focus on the interactions between the Mobile Client, the API Gateway, the Application Tier components, and the Data Tier, highlighting how functional requirements are realized at runtime.

[UC01] Secure Login

This sequence diagram describes the secure login process for a Registered User. The Mobile Client collects user credentials and forwards them to the backend through the API Gateway. The Authentication Manager validates the credentials against the database and, if successful, issues a valid authentication token that is returned to the client. This token is subsequently used to access protected functionalities.

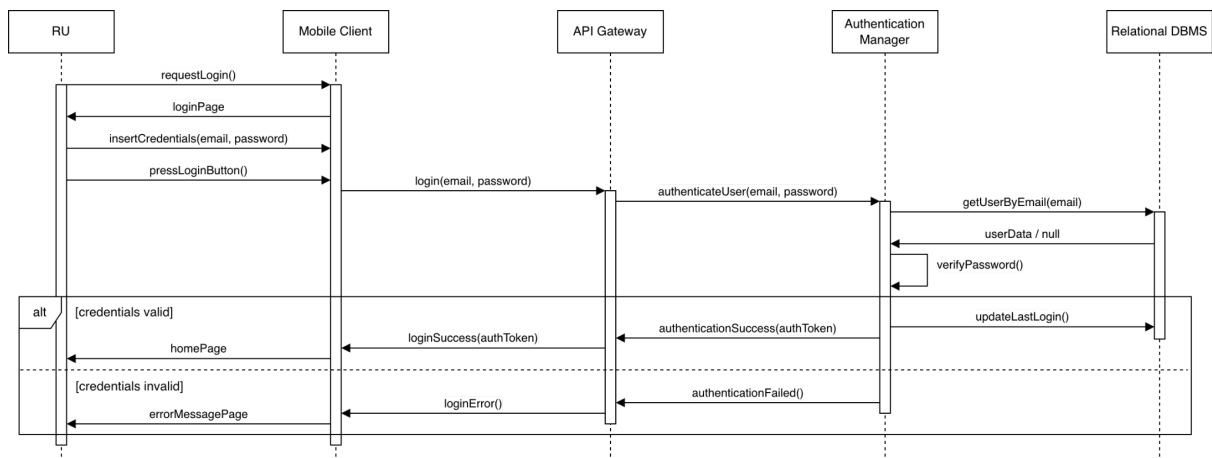


Figure 2.8: Secure Login - Sequence Diagram

[UC02] Start/Stop Trip Recording

This sequence diagram illustrates the lifecycle of an automated trip recording. The Registered User starts and stops a trip through the Mobile Client, which manages real-time sensor data acquisition locally. Upon trip completion, the collected data are sent to the backend, where the Trip Lifecycle Manager persists the trip and triggers the computation of aggregated statistics.

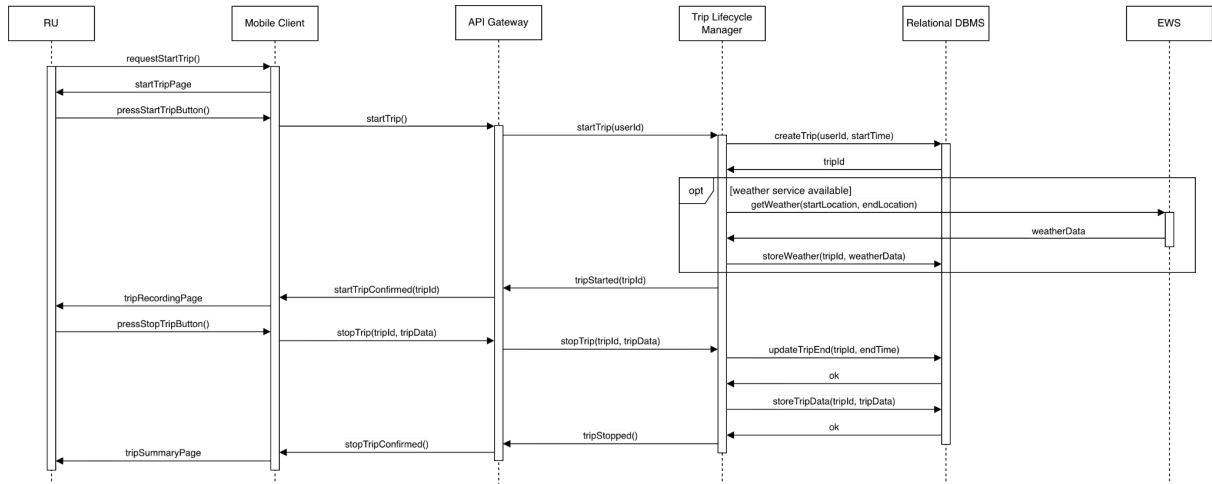


Figure 2.9: Start/Stop Trip Recording - Sequence Diagram

[UC03] Review Auto-Detected Issues

This sequence diagram describes the review and validation workflow for automatically detected anomalies. After trip completion, candidate reports generated by the Auto Detection System are retrieved by the user through the Mobile Client. The user can confirm, correct, or reject each report, and the Review & Validation Manager updates their state accordingly in the database.

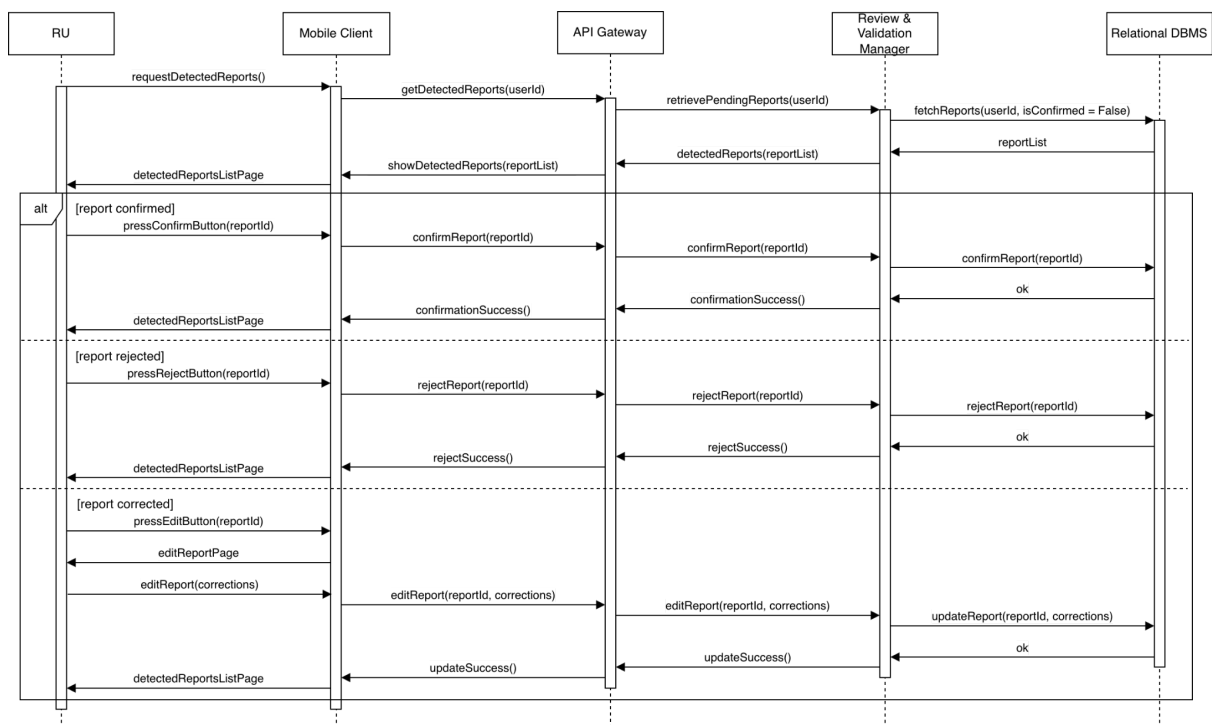


Figure 2.10: Review Auto-Detected Issues - Sequence Diagram

[UCo4] Search and view Paths

This sequence diagram shows how users search and visualize bike paths between an origin and a destination. Candidate routes are obtained through the External Map Service and subsequently ranked by the Recommendation & PathScore Engine based on published reports and path conditions. The resulting ranked paths and overlays are then displayed on the map through the Mobile Client.

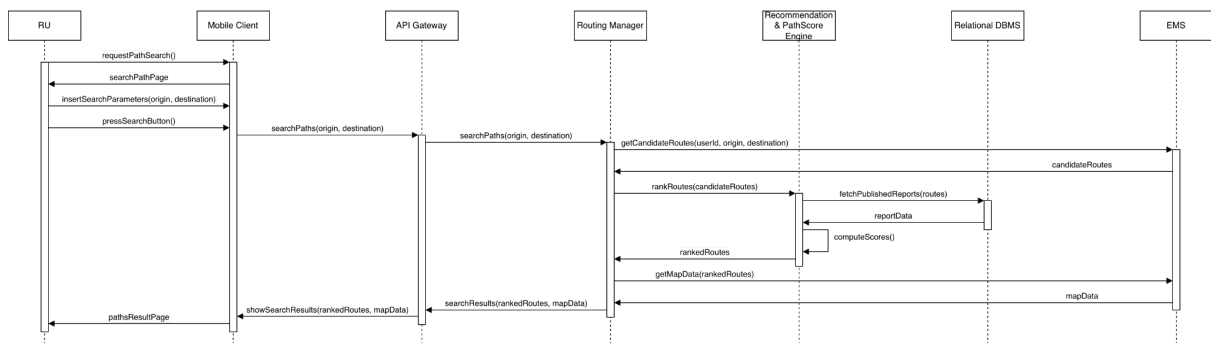


Figure 2.11: Search and View Paths - Sequence Diagram

[UCo5] Manual Data Contribution

This sequence diagram describes the manual insertion of path conditions and obstacles by a Registered User. The Mobile Client sends the provided data to the backend, where the Manual Report Manager validates and stores them as draft reports pending publication.

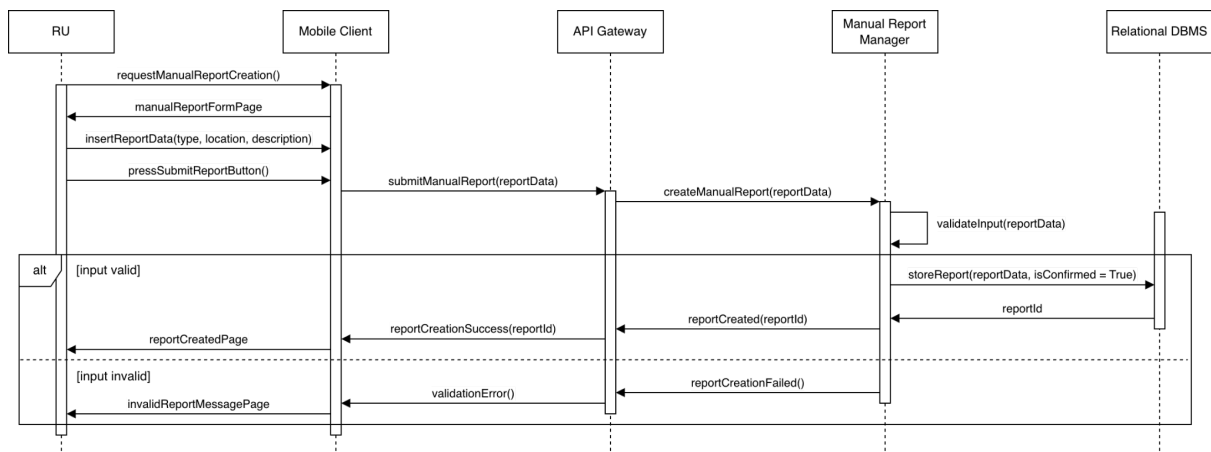


Figure 2.12: Manual Data Contribution - Sequence Diagram

[UCo6] View Trip Details

This sequence diagram illustrates how a Registered User accesses the details of a previously recorded trip. The Mobile Client requests trip data and statistics from the backend, which retrieves the relevant information from the database and returns it for visualization.

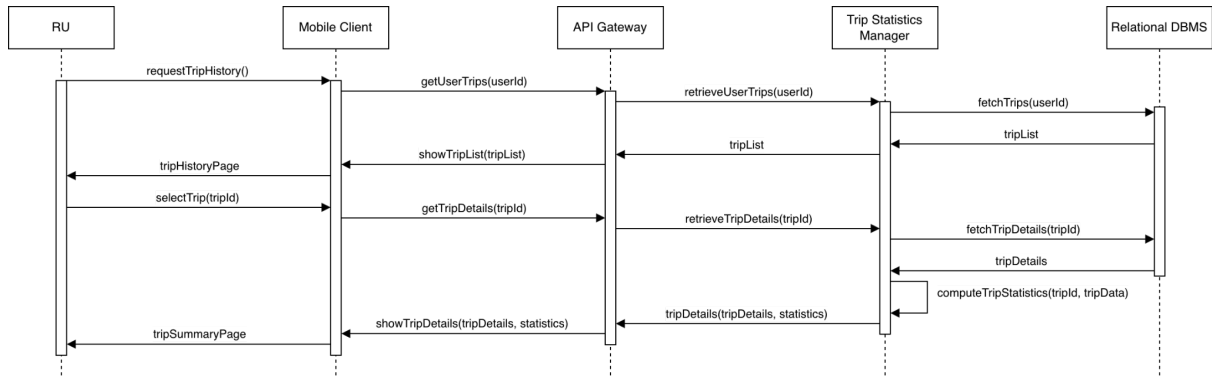


Figure 2.13: View Trip Details - Sequence Diagram

[UCo7] Publish Validated Information

This sequence diagram describes the controlled publication of validated reports. When the user explicitly requests publication, the Publishing Manager verifies ownership and validation status before updating the report state. Only successfully published information becomes publicly accessible and usable for path ranking.

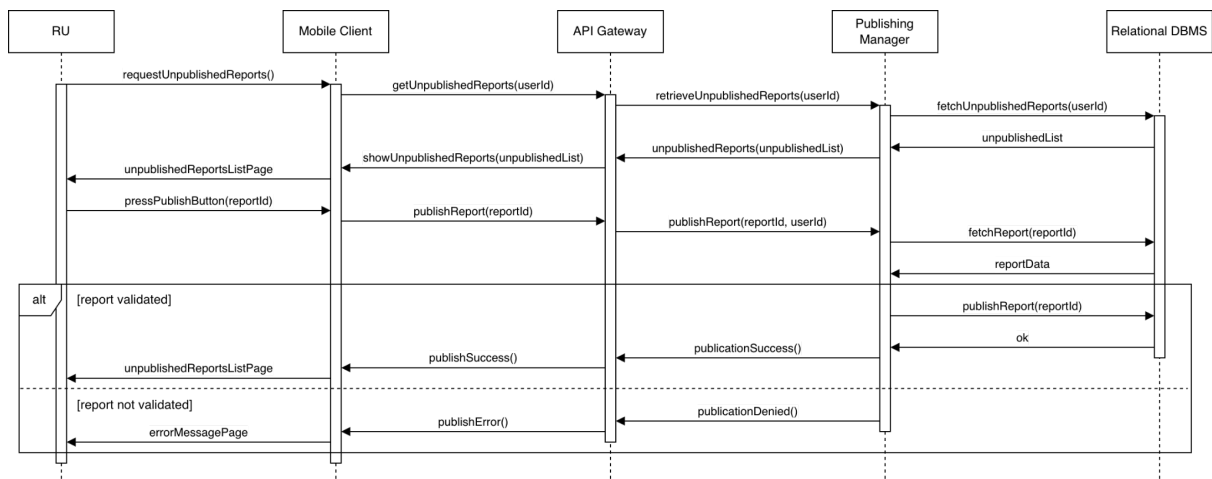


Figure 2.14: Publish Validated Information - Sequence Diagram

2.5. Components Interfaces

API Gateway

- login(email, password)
- startTrip(userId)
- stopTrip(tripId)
- retrieveUserTrip(userId)
- retrieveTripDetails(tripId)
- retrievePendingReports(userId)
- confirmReport(reportId)
- rejectReport(reportId)
- editReport(reportId, corrections)
- retrieveUnpublishedReports(userId)
- publishReport(userId, reportId)

- searchPaths(origin, destination)
- getPathDetails(userIdOrGuest, pathId)
- createManualReport(reportData)

Authentication Manager

- authenticateUser(email, password)
- authenticationSuccess(authToken)
- authenticationFailed()

User Profile Manager

- getProfile(userId)
- updateProfile(userId, profileData)

Trip Lifecycle Manager

- createTrip(userId, startTime)
- updateTripEnd(tripId, endTime)
- storeTripData(tripId, tripData)
- getWeather(startLocation, endLocation)
- storeWeather(tripId, weatherData)

Trip Statistics Manager

- computeTripStatistics(tripId, tripData)
- fetchTrips(userId)
- fetchTripDetails(tripId)

Weather Adapter

- getWeatherData(coordinates, timestamp)

Sensor Data Preprocessor

- preprocessSensorData(rawSensorData)

Anomaly Detection Engine

- detectAnomalies(preprocessedSensorData)

Candidate Report Generator

- createCandidateReports(tripId, anomalies)
- storeCandidateReports(tripId, candidateReports)

Manual Report Manager

- createTrip(userId, segmentId[], status[], obstacleType[], description[])
- validateInput(reportData)
- storeReports(reportData, isConfirmed)

Review & Validation Manager

- fetchReports(userId, isConfirmed)
- confirm(reportId)
- rejectReport(reportId)
- updateReport(reportId, corrections)

Publishing Manager

- fetchUnpublishedReports(userId)
- fetchReport(reportId)
- publishReport(reportId)

Report Editing Manager

- edit_Report(reportId, updatedData)

Routing Manager

- getCandidateRoutes(userId, origin, destination)
- rankRoutes(candidateRoutes)
- getMapData(rankedRoutes)

Scoring Interface

- request_Scoring_Data(pathSegments)

PathScore Calculator

- compute_PathScore(pathSegments, reports)

Ranking Manager

- Rank_Paths(scoredPaths)

Map Adapter

- request_Map_Tiles(area)
- getMapData(rankedRoutes)

Overlay Generator

- generate_Map_Overlay(paths, scores, reports)

Recommendation & PathScore Engine

- fetchPublishedReports(routes)
- computeScores()

External Weather Service (EWS API)

- getWeather(startLocation, endLocation)

External Map Service (EMS API)

- getMapData(rankedRoutes)
- getCandidateRoutes(origin, destination)

Relational DBMS

- getUserByEmail(email)
- createUser(userData)
- createTrip(userId, tripMetadata)
- updateTripEnd(tripId, endTime)
- storeTripTrace(tripId, traceData)
- storeWeather(tripId, weatherData)
- getTrips(userId)
- getTripDetails(tripId)
- storeReport(reportData)

- getReportsByUser(userId, filters)
- getPublishedReportsBySegments(segments)
- updateReport(reportId, updatedData)
- updateReportState(reportId, state)
- updateReportPublication(reportId, isPublished)

Mobile Client

- requestLogin(email, password)
- startTrip()
- stopTrip(tripId, tripData)
- confirmReport(reportId)
- rejectReport(reportId)
- editReport(reportId, corrections)
- searchPath(origin, destination)
- getDetectedReports(userId)
- submitManualReport(reportData)
- getUserTrips(userId)
- getTripDetails(tripId)
- getUnpublishedReports(userId)
- publishReport(reportId)
- requestPathDetails(pathId)

2.6. Data Logic Model

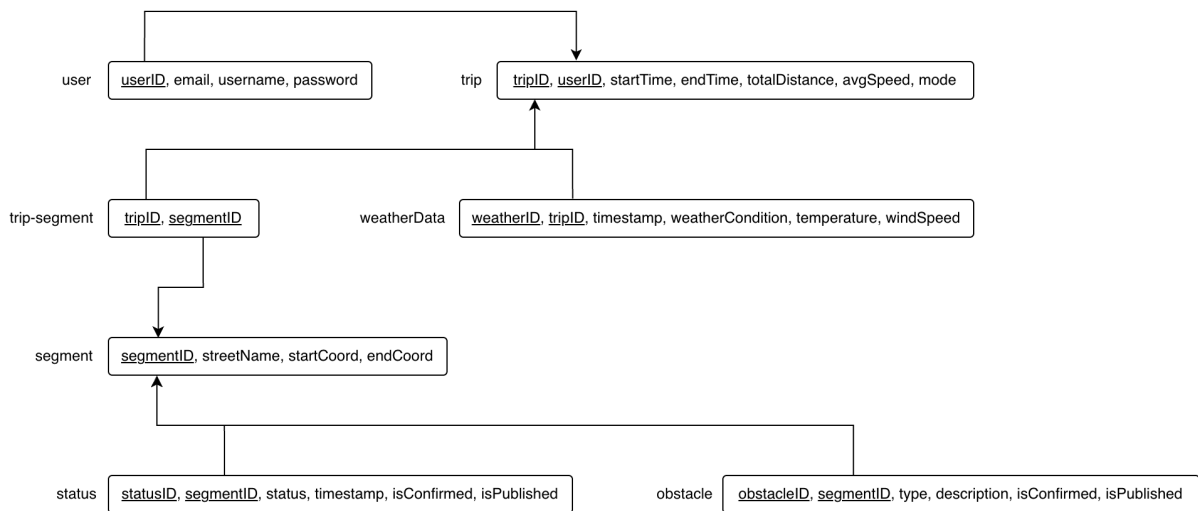


Figure 2.15: Data Logic Model Scheme

The Data Logic Model describes the logical organization of the data managed by the BBP system and is derived from the domain concepts identified in the RASD.

Registered users are represented by the User entity and can record multiple Trips, each storing temporal information and aggregated statistics. A trip is composed of multiple road Segments, modeled through the Trip-Segment association to represent a many-to-many relationship.

Additional contextual information is captured through WeatherData, which enriches trips with meteorological conditions retrieved from an external service. User-generated information on road conditions is represented by Status and Obstacles, both associated with segments and supporting confirmation and publication states.

This model ensures data consistency, avoids redundancy, and supports the core functionalities of trip management, path evaluation, and public information sharing.

2.7. Selected Architectural Style and Patterns

2.7.1. 3-Tier Architecture

As mentioned in Section 2.1, the BBP system is designed according to a 3-tier architecture, which provides clear separation of concerns and supports scalability and modularity.

The Presentation Tier is implemented by the Mobile Client, responsible for user interaction, map visualization, and sensor data acquisition.

The Application Tier hosts the core business logic, including trip management, report validation, routing, and path scoring.

Regarding the Data Tier, a relational database is adopted to manage structured domain data such as users, trips, paths, segments, and reports.

2.7.2. Model-View-Controller (MVC) pattern

Since BBP is designed to be delivered as an application with a clear separation between user interface and backend logic, the Model-View-Controller (MVC) pattern was adopted to structure the system.

- **Model:** includes the domain entities and application data (e.g., User, Trip, PathSegment, ObstacleReport), together with the business logic services that operate on them (e.g., validation rules and PathScore computation).
- **View:** corresponds to the user interface rendered by the Mobile Client, responsible for presenting maps, trips, reports, and route rankings in a clear and usable way.
- **Controller:** includes the components that receive user actions and translate them into system operations, typically represented by the API Gateway and the service endpoints handling the request flow.

The adoption of MVC improves maintainability and testability by isolating the UI from the core system logic, allowing independent development and easier functional testing of the business rules (e.g., validation workflow, publication constraints, and ranking logic).

2.7.3. Service-Oriented Decomposition within the Application Tier

In addition to the overall 3-tier structure, the Application Tier has been decomposed into cohesive backend services (macro-components) such as the Auto Detection System, Report & Validation System, and Recommendation & PathScore Engine. This decomposition supports independent scaling and performance isolation for computationally intensive tasks, improving responsiveness of user-facing operations.

3. User Interface Design

While in the RASD we described the main UI requirements and interaction principles, in this section we summarized the navigation structure and the key pages of the BBP application. Moreover, we also specify how the User Interfaces section of the RASD map onto the pages effectively designed (Section 3.3).

Public pages are accessible without authentication and allow Guest Users to consult public information:

- Login pages:
 - WelcomePage
 - LoginPage
- RegistrationPage
- Path Search pages:
 - PathSearchPage
 - PathResultPage
 - PathDetailsPage

Private pages are only accessible with authentication by Registered Users to contribute informations on the platform:

- HomePage
- Trip Recording pages:
 - TripRecordingPage
 - TripSummaryPage
 - TripHistoryPage
- Manual Report pages:
 - ManualReportFormPage
 - InvalidReportMessagePage
- Review Auto-Detected Issues pages:
 - DetectedReportsListPage
 - EditReportPage
- UnpublishedReportsListPage
- ProfileSettingsPage
- Utility pages:
 - LoadingPage
 - ErrorMessagePage

3.1. Public Pages

3.1.1. Public Pages Navigation Map

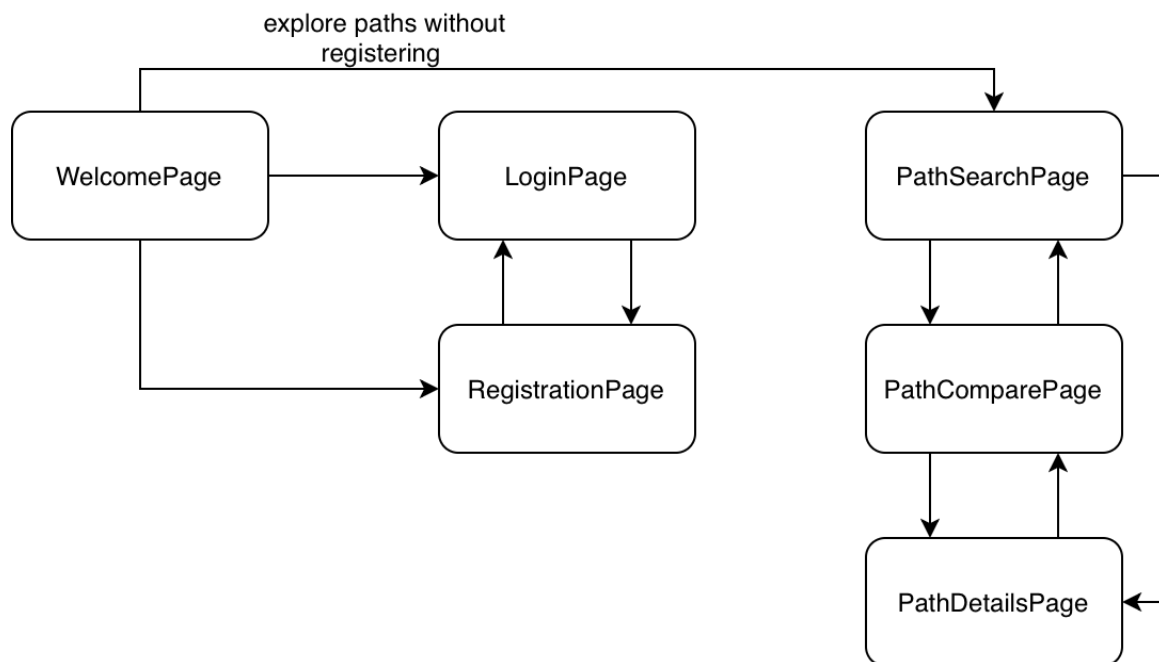


Figure 3.1: Public Pages Navigation Map

3.1.2. Login Pages Mock-up

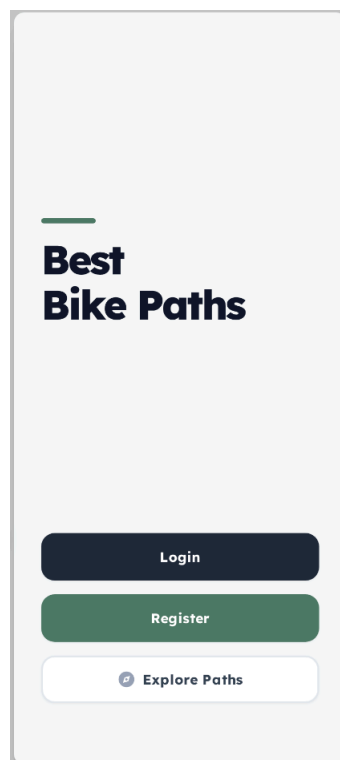
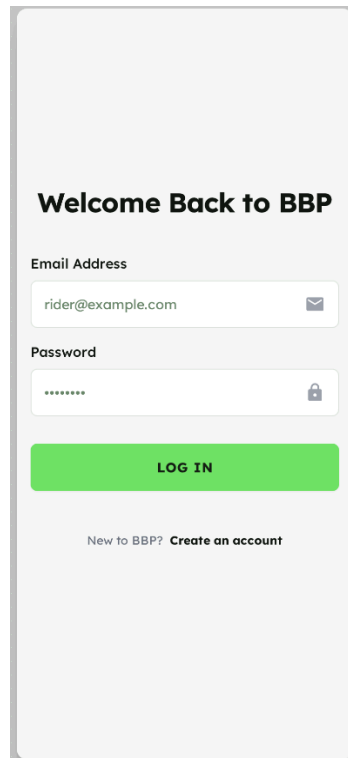


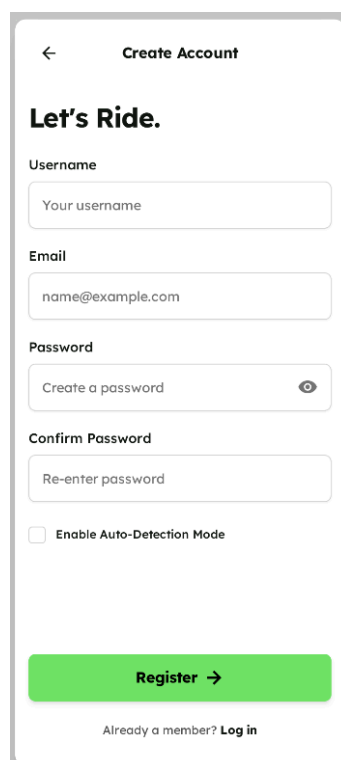
Figure 3.2: WelcomePage Mock-up



A mobile app login screen with a light gray background. At the top, the text "Welcome Back to BBP" is centered in bold. Below it, there are two input fields: "Email Address" with the placeholder "rider@example.com" and an envelope icon, and "Password" with a lock icon. A green "LOG IN" button is centered below the fields. At the bottom, there is a link "New to BBP? Create an account".

Figure 3.3: LoginPage Mock-up

3.1.3. Registration Page Mock-up



A mobile app registration screen with a white background. At the top, there is a back arrow and the text "Create Account". Below it, the text "Let's Ride." is centered in bold. There are four input fields: "Username" with placeholder "Your username", "Email" with placeholder "name@example.com", "Password" with placeholder "Create a password" and an eye icon, and "Confirm Password" with placeholder "Re-enter password". Below the fields is a checkbox labeled "Enable Auto-Detection Mode". A green "Register →" button is centered below the checkbox. At the bottom, there is a link "Already a member? Log in".

Figure 3.4: RegistrationPage Mock-up

3.1.4. Search Path Pages Mock-up

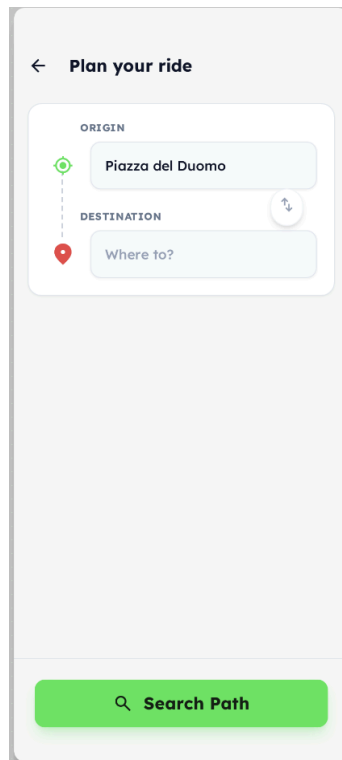


Figure 3.5: SearchPathPage Mock-up

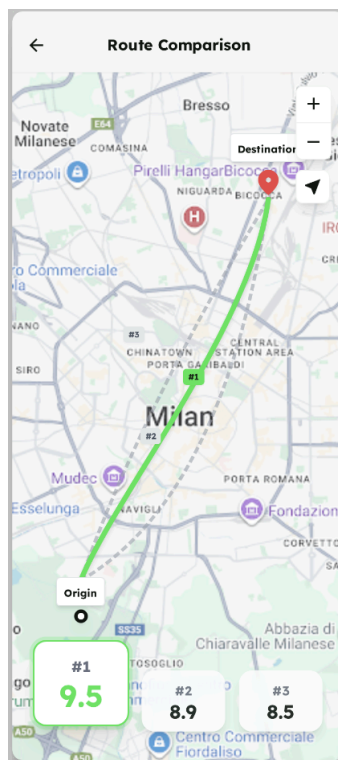


Figure 3.6: PathResultPage Mock-up

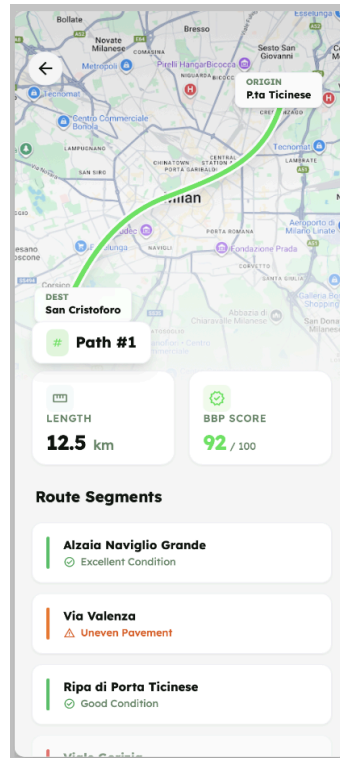


Figure 3.7: PathDetailsPage Mock-up

3.2. Private Pages

3.2.1. Private Pages Navigation Map

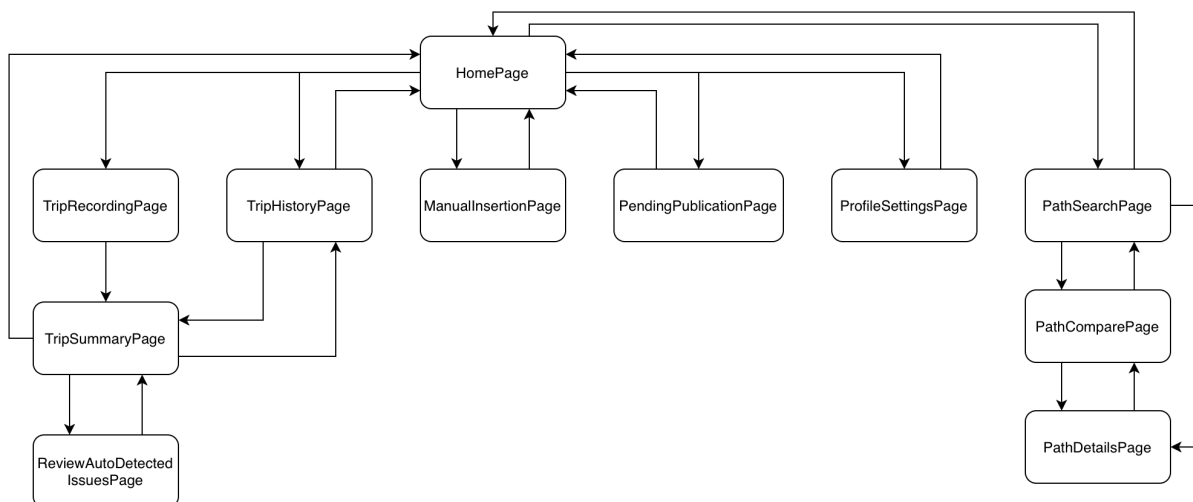


Figure 3.8: Private Pages Navigation Map

3.2.1. Home Page Mock-up

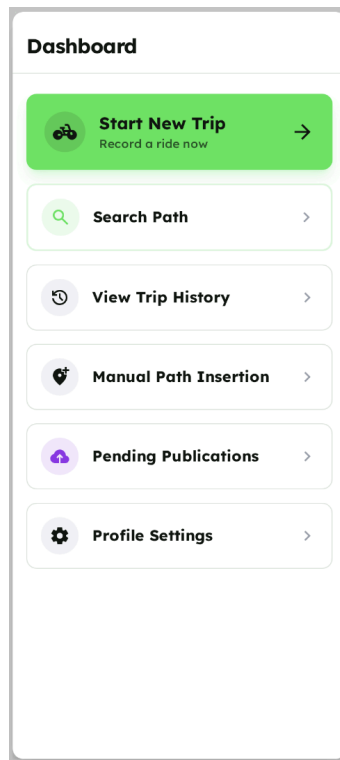


Figure 3.9: HomePage Mock-up

3.2.2. Trip Recording Pages Mock-up

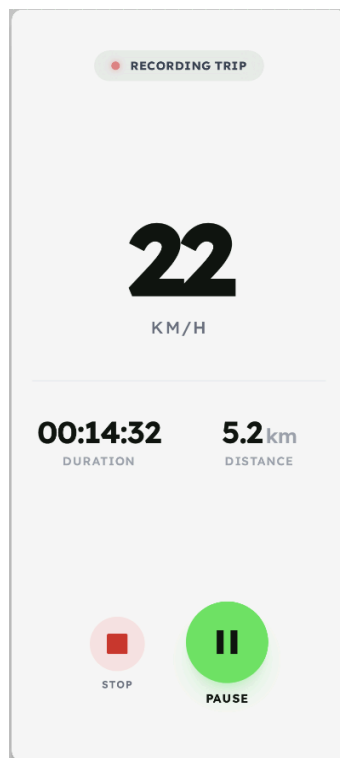


Figure 3.10: TripRecordingPage Mock-up

3. User Interface Design

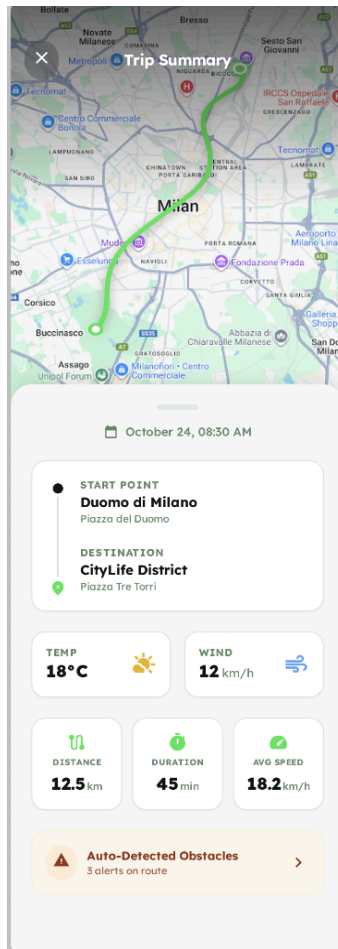


Figure 3.11: TripSummaryPage Mock-up

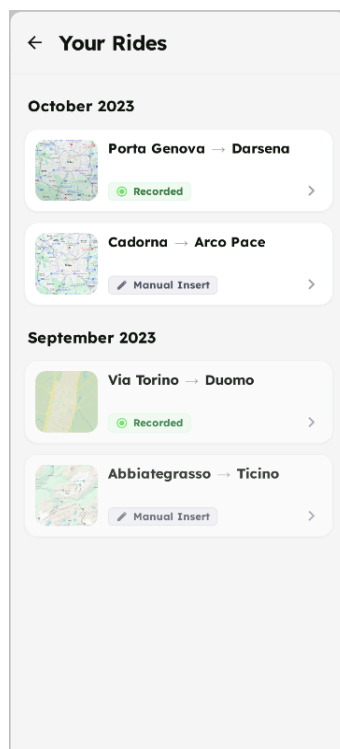
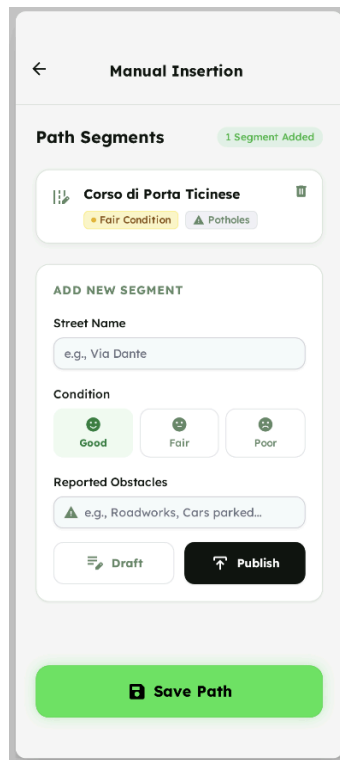


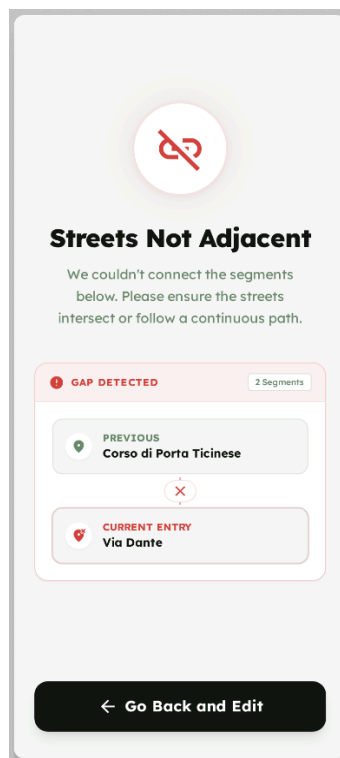
Figure 3.12: TripHistoryPage Mock-up

3.2.3. Manual Report Pages Mock-up



The 'Manual Insertion' form is designed for adding new path segments. It features a back arrow and a title 'Manual Insertion'. Under the 'Path Segments' header, a green badge indicates '1 Segment Added'. The existing segment, 'Corso di Porta Ticinese', is shown with a location pin icon, a 'Fair Condition' tag, and a 'Potholes' tag. The 'ADD NEW SEGMENT' section includes a 'Street Name' input field with a placeholder 'e.g., Via Dante', a 'Condition' section with three radio buttons ('Good', 'Fair', 'Poor') where 'Good' is selected, and a 'Reported Obstacles' input field with a placeholder 'e.g., Roadworks, Cars parked...'. At the bottom of this section are 'Draft' and 'Publish' buttons. A large green 'Save Path' button is at the very bottom.

Figure 3.13: ManualReportFormPage Mock-up



The 'Invalid report message' page displays an error. At the top is a red circular icon with a crossed-out path. The title 'Streets Not Adjacent' is followed by the message: 'We couldn't connect the segments below. Please ensure the streets intersect or follow a continuous path.' Below this, a red-bordered box contains a 'GAP DETECTED' header and a '2 Segments' badge. It lists the 'PREVIOUS' segment as 'Corso di Porta Ticinese' and the 'CURRENT ENTRY' as 'Via Dante', with a red 'X' between them. A black 'Go Back and Edit' button is at the bottom.

Figure 3.14: InvalidReportMessagePage Mock-up

3.2.4. Review Auto-Detected Issues Pages Mock-up

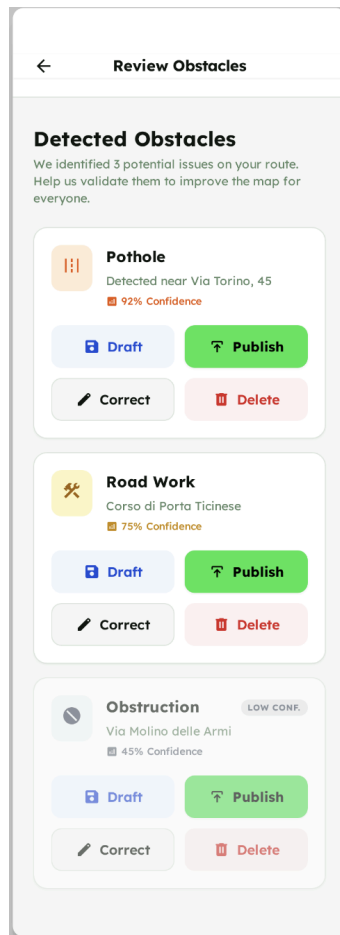


Figure 3.15: DetectedReportsListPage Mock-up

Review Obstacles

Detected Obstacles

We identified 3 potential issues on your route.
Help us validate them to improve the map for everyone.

Correction Mode

92% Confidence

OBSTACLE TYPE

Pothole

LOCATION

Via Torino, 45

DESCRIPTION

Add details about the obstacle...

Cancel

Save Changes

Road Work

Corso di Porta Ticinese

75% Confidence

Correct

Delete

Obstruction

LOW CONF.

Via Molino delle Armi

45% Confidence

Correct

Delete

Figure 3.16: EditReportPage Mock-up

29

3.2.5. Unpublished Reports Page Mock-up

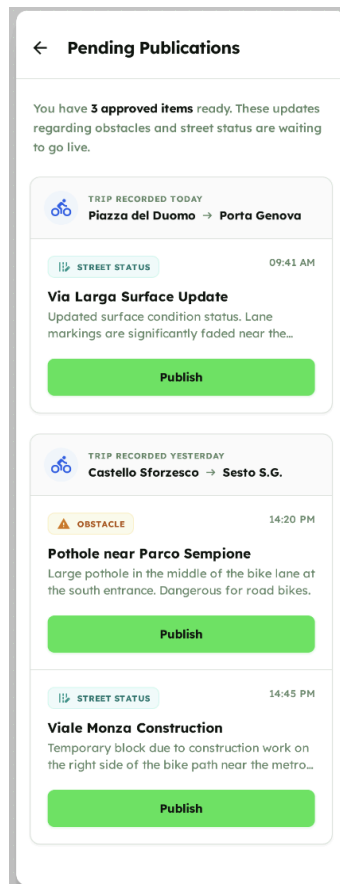


Figure 3.17: UnpublishedReportsListPage Mock-up

3.2.6. Profile Settings Page Mock-up

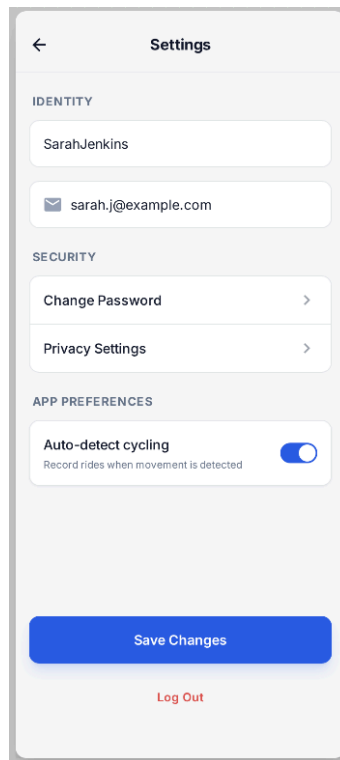


Figure 3.18: ProfileSettingsPage Mock-up

3.2.7. Utility Pages Mock-up

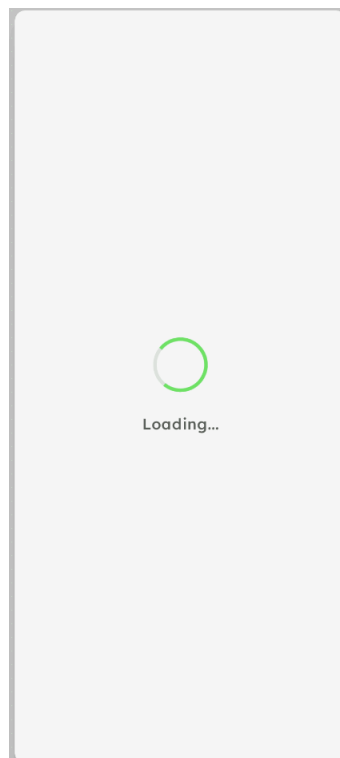


Figure 3.19: LoadingPage Mock-up

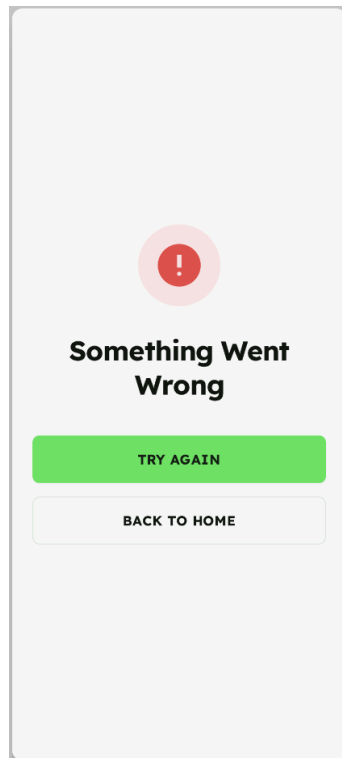


Figure 3.20: ErrorMessagePage Mock-up

3.3. Requirements UI Sections Mapping

The following mapping shows how the UI capabilities described in the RASD are implemented through the pages designed in the DD.

RASD UI Section	DD Pages
Access and registration	WelcomePage, LoginPage, RegistrationPage
Real-time trip recording and live indicators	TripRecordingPage
Visualization of recorded path and trip statistics	TripHistoryPage, TripSummaryPage
Interactive map exploration of paths and reports	PathSearchPage, PathResultPage, PathDetailsPage
Manual insertion of path, streets conditions and obstacles	ManualReportFormPage
Review and validation of automatically detected issues	DetectedReportsListPage, EditReportPage
Draft management and controlled publication	UnpublishedReportsListPage

Profile Management	ProfileSettingsPage
System Utility Pages	LoadingPage, ErrorMessagePage

Table 3.1: Requirements UI Sections Mapping

4. Requirements Traceability

4.1. Registered User Goals

G01 – Account creation and secure login

- Requirements: R01
- Design elements: MC (login/registration UI), API GW (authentication endpoint), U&T (user management), DBMS

G02 – Start, pause, and stop automated trip recording

- Requirements: R02, R10
- Design elements: MC (sensor acquisition, trip control), API GW (sync endpoints), U&T (trip session management), DBMS

G03 – Store completed trips in personal history

- Requirements: R03
- Design elements: MC, API GW, U&T (trip persistence), DBMS

G04 – View trip statistics

- Requirements: R04, R19
- Design elements: MC (visualization), U&T (statistics aggregation), DBMS

G05 – Enrich trip data with weather information

- Requirements: R05
- Design elements: U&T, EWS, DBMS, MC (data visualization)

G06 – Manually insert a path street by street

- Requirements: R06, R07
- Design elements: MC (manual insertion UI), API GW, R&V (validation), DBMS, EMS

G07 – Manually report street conditions and obstacles

- Requirements: R08, R09
- Design elements: MC, API GW, R&V, DBMS, PSE (score update)

G08 – Activate automated obstacle detection

- Requirements: R10, R11
- Design elements: MC (sensor acquisition), ADS (data analysis), DBMS

G09 – Review and validate automatically detected obstacles

- Requirements: R12
- Design elements: MC (review UI), ADS, R&V (workflow/state handling), DBMS

G10 – Save path information as draft

- Requirements: R13
- Design elements: MC, API GW, R&V (draft state), DBMS

G11 – Publish validated path information

- Requirements: R14
- Design elements: MC, API GW, R&V (state transition), DBMS

G12 – Search bike paths between origin and destination

- Requirements: R15, R19
- Design elements: MC, API GW, RMI, EMS, DBMS

G13 – Compare and visualize ranked paths

- Requirements: R16, R17, R19
- Design elements: MC, RMI, PSE, DBMS, EMS

4.2. Guest User Goals

G14 – Search bike paths without registration

- Requirements: R15, R18, R19
- Design elements: MC, API GW, RMI, EMS, DBMS

G15 – View ranked paths and public reports

- Requirements: R16, R17, R18, R19
- Design elements: MC, RMI, PSE, DBMS, EMS

5. Implementation, Integration and Test Plan

5.1. Overview

This chapter describes the strategy adopted for the implementation, integration, and testing of the Best Bike Paths (BBP) system.

The goal of this plan is to ensure that all system components are implemented in a structured manner, progressively integrated, and thoroughly tested to verify compliance with the functional and non-functional requirements defined in the RASD.

Testing plays a crucial role in the development process, as BBP relies on sensor data, user validation, and aggregated community information. Therefore, both individual component correctness and correct interaction between components must be ensured before system release.

5.2. Implementation Plan

The implementation of BBP follows a top-down approach, starting from the foundational components required to enable user interaction and data persistence, and progressively introducing more complex and computation-intensive functionalities.

Each component is implemented and tested independently before being integrated into the overall architecture.

This strategy allows parallel development and reduces integration risks by identifying defects at an early stage.

5.2.1. Feature Identification

Starting from the product functions and requirements defined in the RASD, the following core features have been identified as the main implementation milestones:

F1 - User registration and authentication

This feature ensures that Registered Users can create accounts and securely authenticate. It is a prerequisite for accessing all personalized functionalities such as trip recording, reporting, and publication.

F2 - Trip recording and personal trip management

This feature includes real-time GPS acquisition on the mobile device, trip lifecycle management (start, pause, stop), storage of completed trips, and visualization of personal statistics.

F3 - Automated data acquisition and obstacle detection

This feature enables the Automated Mode, where sensor data (GPS, accelerometer, gyroscope) is collected on the mobile client and analyzed after trip completion to detect potential anomalies or obstacles.

F4 - Manual and automatic reporting with validation workflow

This feature supports manual insertion of path conditions and obstacles, as well as the review, confirmation, or rejection of automatically detected issues before publication.

F5 - Path search, ranking, and visualization

This is a core system feature allowing both Registered and Guest Users to search for bike paths, compare alternative routes, and visualize ranked results based on Path Score and published reports.

F6 - Data publication and community access

This feature manages the controlled publication of validated data and its integration into the shared dataset used for routing and scoring.

5.3. Component Integration and Testing

This section describes the recommended order for component development, integration, and validation.

Mobile Client and Data Layer

The first elements to be implemented are:

- the Mobile Client, enabling user interaction, GPS and sensor data acquisition, and local buffering;
- the Relational DBMS, responsible for storing users, trips, path segments, reports, and publication states.

These two elements can be developed in parallel, as they define the boundaries for data exchange and persistence.

Once the presentation layer and data layer have been implemented, the application layer can be developed. The components should be developed and tested in the following order:

1. User and Trip Management

The first backend components to be implemented and tested are:

- API Gateway
- User & Trip Management System

These components enable authentication, trip lifecycle management, and storage of personal data. Their early implementation allows basic end-to-end system interaction.

2. Reporting and Validation Components

Once trip management is stable, the following components are introduced:

- Auto Detection System, operating on persisted trip data to identify potential anomalies;
- Report & Validation System, responsible for enforcing the mandatory validation workflow before publication.

These components must be integrated and tested together, as they jointly guarantee data reliability and correctness.

3. Routing, Path Scoring, and Recommendation

After validated data can be stored and published, the system integrates:

- Routing & Map Integration System
- Recommendation & PathScore Engine

These components depend on the availability of published path data and are therefore introduced after reporting and validation mechanisms are fully operational.

4. External Services Integration

Finally, integration with external services is completed:

- External Map Service (EMS) for map visualization and routing support;
- External Weather Service (EWS) for trip data enrichment.

These integrations are tested with fault-injection scenarios to verify system robustness in case of partial service unavailability.

5.4. System Testing

System testing aims to verify that BBP satisfies all requirements specified in the RASD.

Testing is carried out at different levels:

- **Unit Testing:**
Each component is tested independently to verify correctness of internal logic (e.g., Path Score computation, report state transitions).
- **Integration Testing:**
Groups of components are tested together to verify correct interaction (e.g., Auto Detection System with Report & Validation System).
- **Functional Testing:**
The system is executed following the use cases defined in the RASD to verify that all functional requirements are satisfied.
- **Performance Testing:**
The system is tested under realistic and peak workloads to evaluate response times looking for bottlenecks that affect in a negative way the time response of the system.
- **Usability Testing:**
User interactions are evaluated to ensure clarity and ease of use, particularly for reporting, validation, and map-based navigation.

Alpha and Beta Testing

Before release, BBP undergoes staged user testing:

- **Alpha Testing:**
Conducted internally by developers and project stakeholders to identify critical bugs and architectural issues.
- **Beta Testing:**
Conducted with a limited group of external users (cyclists) in real-world conditions to collect feedback on usability, data accuracy, and system performance.

Feedback from both phases is used to refine the system and improve overall reliability before the official release.

6. Effort spent

Group Member	Effort Spent	
Lorusso Luca (10831296)	Introduction	1 h
	Architectural Design	3 h
	User Interface Design	5 h
	Requirements Traceability	3 h
	II&T	5 h
	Reasoning	5 h
Martorano Gabriele (10822443)	Introduction	2 h
	Architectural Design	6 h
	User Interface Design	4 h
	Requirements Traceability	2 h
	II&T	3 h
	Reasoning	5 h

Table 6.1: Member Effort Spent table

7. References

7.1. References

- Assignment RDD AY 2025-2026
- github - LorussoMartorano

7.2. Used Tools

- GitHub – Used for version control and collaborative development
- draw.io – Used for UML, architectural and deployment diagrams
- Adobe Xd – Used for Mock-up design
- ChatGPT:
 - **Inputs provided:** Draft sections of the DD
 - **Outputs obtained:** Suggestions for text refinement, summarization, and improved consistency of descriptions
 - **Verification:** All generated content was manually reviewed, validated, and integrated by the authors to ensure correctness and coherence with the project specification